Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Kontrol Akışı - Döngüler

- Iterative statement (döngü):
  - Bir veya birden fazla komutun tekrar tekrar çalıştırılmasını sağlayan kontrol yapısıdır.
  - Döngüler:0 kez 1 kez veya birçok kez çalışabilir.
  - Döngüler programlamanın temelidir çünkü:
    - Aynı işlemi tekrar yazmayı engeller.
    - Kodun kısa ve esnek olmasını sağlar.
    - Eğer döngüler olmasaydı her işlem tek tek yazılmak zorunda olurdu.
    - Programlar çok uzun, bakımı zor ve bellek açısından verimsiz olurdu.

```c
a[0] = a[0] * 2;
a[1] = a[1] * 2;
a[2] = a[2] * 2;
...
```

```c
for (i = 0; i < n; i++) {
    a[i] = a[i] * 2;
}
```

Döngü nasıl kontrol edilir?

Sayma (counter-controlled)

Mantıksal (logical-controlled)

Kombinasyon

Kontrol nerede yapılır?

Döngü başında (pretest)

Döngü sonunda (posttest)

Kontrol Akışı - Döngüler

- Pretest (Ön kontrol)
  - Döngü başlamadan kontrol yapılır.
  - Eğer şart false → hiç çalışmaz.

```c
while (x > 0) {
    x--;
}
```

- Posttest (Son kontrol)
  - Döngü en az 1 kez çalışır.
    - while → 0 veya daha fazla çalışır.
    - do-while → en az 1 kez çalışır.

```c
do {
    x--;
} while (x > 0);
```

Kontrol Akışı - Döngüler

- Sayaçlı Döngüde döngü bir sayaca göre çalışır.
  - 3 temel parametre:
    - Başlangıç değeri
    - Bitiş değeri
    - Artış miktarı (step)

```c
for (int i = 1; i <= 10; i++) {
    printf("%d\n", i);
}
```

- Döngü değişkeni tipi ne olmalı?
  - int, float, char?
  - Döngü içinde değiştirilebilir mi?
  - Parametreler kaç kez değerlendirilmeli?
    - 1 kez → basit ama katı
    - her iterasyonda → esnek ama karmaşık

```c
for (i = 0; i < 10; i++) {
    i = i + 2; // kafa karıştırıcı!
}
```

Kontrol Akışı - Döngüler

- Python for Döngüsü

```python
for x in range(5):
    print(x)
```

- C for döngüsü

```c
for (i = 0; i < 5; i++) {
    printf("%d\n", i);
}
```

- Ada for döngüsü

```ada
for Count in 1..10 loop
   Sum := Sum + Count;
end loop;
```

- Python foreach

```python
for x in [2,4,6]:
    print(x)
```

Kontrol Akışı - Döngüler

- Fonksiyonel dillerde döngü yoktur. → özyineleme (tail-recusion) kullanılır.

```fsharp
let rec loop n =
    if n <= 0 then ()
    else
        printfn "%d" n
        loop (n-1)
```

- Döngüler programlamanın temelidir.
- 3 ana türü vardır.
  - Counter-controlled. (for loop)
  - Logical-controlled (while)
  - Data-based (foreach)
- Önemli kavramlar:
  - pretest / posttest
  - break / continue
  - recursion
- ✓ İyi tasarım:
  - Okunabilirlik,
  - Basitlik
  - Hata azaltma

Kontrol Akışı – Guarded Command

- Bir komutun çalışması için bir koşulun (guard) sağlanması gerekir.
- Genel yapı:
  - guard → statement
    - Eğer guard (koşul) true ise → statement çalışır.
    - Eğer false ise → çalışmaz
- Guarded Loop (Tekrarlı Yapı)

```
[] q3 > q4 -> temp := q3;
              q3 := q4;
              q4 := temp;
od
```

- Dijkstra ve Guarded Commands.
- Bu yapı, bilgisayar bilimci Edsger W. Dijkstra tarafından geliştirilmiştir.
- Amaç programların daha matematiksel, daha doğrulanabilir (verifiable), daha deterministik olmayan (nondeterministic) olmasını sağlamaktır.
- Program sadece çalıştırılmak için değil, ispatlanabilir (provable) olmalıdır.

Kontrol Akışı – Guarded Command

- Guarded Loop (Tekrarlı Yapı)

```
[] q3 > q4 -> temp := q3;
              q3 := q4;
              q4 := temp;
od
```

- [] → alternatif seçim (birden fazla guard olabilir)
  - q3 > q4 → guard (koşul)
  - → koşul sağlanırsa çalışacak blok.
  - od → döngünün sonu
- ✓ Anlamı:q3 > q4 olduğu sürece:q3 ve q4 yer değiştirir.

- Dijkstra ve Guarded Commands.
- Bu yapı, bilgisayar bilimci Edsger W. Dijkstra tarafından geliştirilmiştir.
- Amaç programların daha matematiksel, daha doğrulanabilir (verifiable), daha deterministik olmayan (nondeterministic) olmasını sağlamaktır.
- Program sadece çalıştırılmak için değil, ispatlanabilir (provable) olmalıdır.

Kontrol Akışı – Guarded Command

```
[] q3 > q4 -> swap(q3, q4)
od
```

- **İşleyiş:**
  - q3 > q4 kontrol edilir
  - Eğer doğruysa:
    - q3 ve q4 yer değiştirir
  - Tekrar kontrol edilir
    - Koşul false olana kadar devam eder
  - **Sonuç:**
    - Döngü sonunda q4 >= q3/

Soru

- Yanda verilen Java programını goto ve break olmadan yazınız.

```
j = -3;
for (i = 0; i < 3; i++) {
  switch (j + 2) {
    case 3:
    case 2: j--; break;
    case 0: j += 2; break;
    default: j = 0;
  }
  if (j > 0) break;
  j = 3 - i
}
```

Soru

```
i = 0;

while (i < 3 && j <= 0) {

    // switch yerine if-else
    if (j + 2 == 3 || j + 2 == 2) {
        j--;
    }
    else if (j + 2 == 0) {
        j = 2;
    }
    else {
        j = 0;
    }

    // break yerine loop condition kullanıldı
    if (j <= 0) {
        j = 3 - i;
        i++;
    }
}
```

- for döngüsü yerine while kullanımı.
- switch yerine if-else-if yapısı